

Drone Delivery Hangar

by Miles Porter

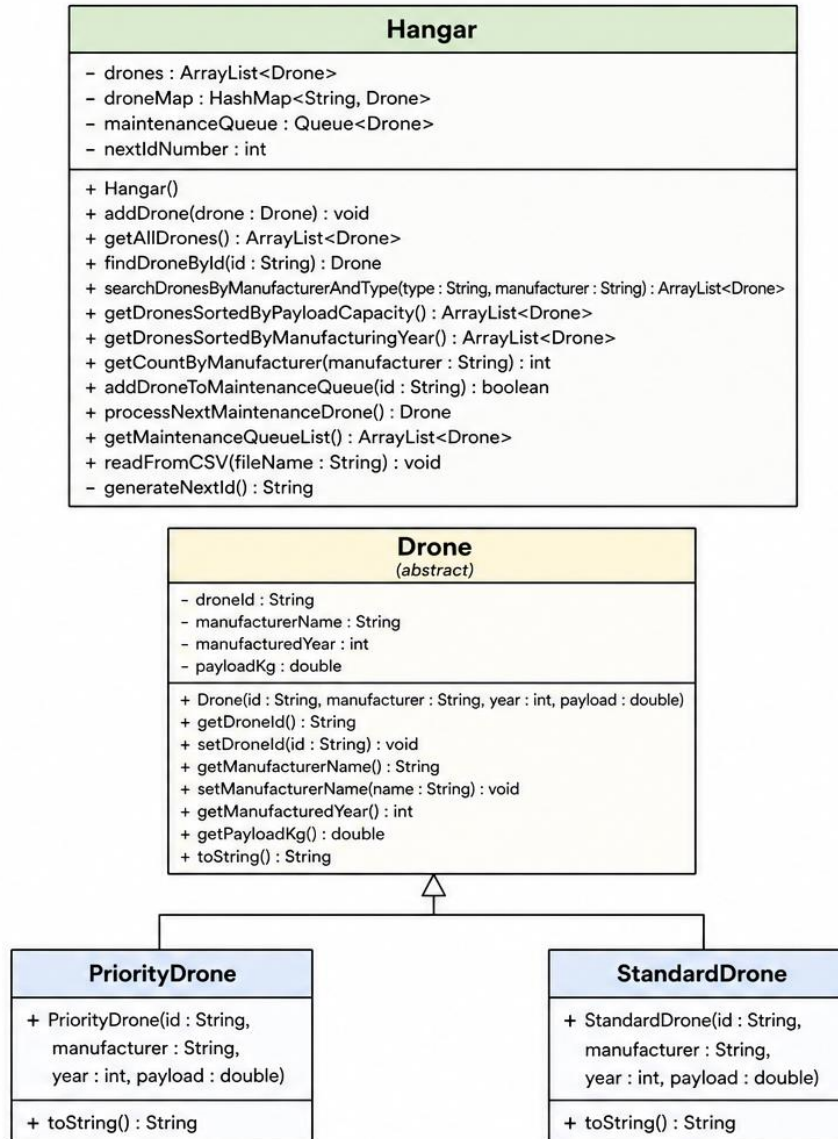
Technical Documentation and JUnit Test Plan

1. SYSTEM OVERVIEW	2
KEY FEATURES	3
UML DESIGN	2
2. DATA STRUCTURE DECISIONS	3
ARRAYLIST<DRONE>	3
HASHMAP<STRING, DRONE>	3
QUEUE<DRONE> USING LINKEDLIST	3
3. DATA STRUCTURE COMPARISON: ARRAYLIST-BASED QUEUE VS QUEUE USING LINKEDLIST	3
VISUAL REPRESENTATION	4
4. UNIQUE ID IMPACT	4
5. JUNIT TEST CASE TABLES	5
HANGAR CLASS TESTS	5
DRONE CLASS TESTS	8
PRIORITYDRONE CLASS TESTS	9
STANDARDDRONE CLASS TESTS	9

1. System Overview

The Drone Delivery Hangar program is a menu-driven Java application that manages a delivery drone inventory. The program can load drones from a CSV file, display the inventory, search by manufacturer and drone type, sort by payload capacity or manufacturing year, count drones by manufacturer, find a drone by unique ID, and process drones through a maintenance queue.

UML Design



Key Features

- CSV loading with validation for drone type, manufacturer, manufacturing year, and payload capacity
- Unique drone IDs using the format D1000, D1001, D1002, etc
- Inventory storage using `ArrayList<Drone>`
- Fast drone lookup using `HashMap<String, Drone>`
- Maintenance processing using `Queue<Drone>` with `LinkedList`
- JUnit tests cover normal behavior, edge cases, and empty-result cases

2. Data Structure Decisions

`ArrayList<Drone>`

An `ArrayList` is a resizable list that stores objects in order. In this program, the `ArrayList` stores the full drone inventory. It is used when displaying all drones, searching through the inventory, counting drones by manufacturer, and creating sorted copies of the inventory. This structure is useful because the number of drones is not known ahead of time and the program often needs to loop through every drone.

`HashMap<String, Drone>`

A `HashMap` stores key-value pairs. In this program, the drone ID is the key and the `Drone` object is the value. This allows the system to quickly find one exact drone by ID instead of looping through the whole `ArrayList`. The `HashMap` is used by the find-by-ID feature and by the maintenance queue feature before adding a drone to the queue.

`Queue<Drone>` using `LinkedList`

A `Queue` is a first-in, first-out data structure. In this program, the `Queue` stores drones waiting for maintenance. The first drone added to maintenance is the first drone processed. A `LinkedList` works well as the `Queue` implementation because it naturally supports adding to the back and removing from the front.

3. Data Structure Comparison: `ArrayList`-Based Queue vs Queue Using `LinkedList`

In the lab customer service queue, we used an `ArrayList<Customer>` to store customers. Customers were added with `queue.add(customer)`, which placed each new customer at the end of the list. However, the lab's `dequeue()` method removed the customer at `queue.size() - 1`, which is the last customer in the list. This means the lab version did not behave like a true first-in, first-out queue. Instead, it behaved more like a last-in, first-out structure because the newest customer was served first.

In my Drone Delivery Hangar project, I used `Queue<Drone>` with a `LinkedList` for the maintenance workflow. This was a better fit because Java's `Queue` interface is designed for first-in, first-out behavior. In my program, drones are added to the back of the queue using `maintenanceQueue.add(drone)`, and the next drone is processed using `maintenanceQueue.poll()`. This means the first drone added to the maintenance queue is the first drone removed for service.

The biggest difference I noticed is that the `ArrayList` version required me to manually manage indexes. In the lab, removing `queue.size() - 1` caused the program to remove the newest item first. If I wanted to make that lab behave like a real queue, I would need to remove index 0 instead. In the Drone Delivery Hangar project, I did not need to manage indexes directly. The `Queue` methods made the intent clearer and helped the maintenance feature match the real-world idea of drones waiting their turn for maintenance.

Visual Representation

Queue Comparison: Lab ArrayList vs. Project Queue with LinkedList

LAB: ArrayList Implementation (Behaves like a STACK – LIFO)

How it works in the lab:

- Customers are added to the end of the ArrayList.
- `dequeue()` removes the item at index `size - 1` (the last item).

Step-by-Step Example

Step 1: Add Alice



Step 2: Add Bob



Step 3: Add Charlie



Items are
ADDED
to the **END**

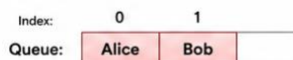
`dequeue()` in Lab

- Removes item at index `size - 1` (last item).

Before `dequeue()`:



After `dequeue()`:



Removed

Result: The **LAST** customer added is served **FIRST**.
This is Last-In, First-Out (LIFO) – behaves like a **STACK**.

PROJECT: Queue with LinkedList (True QUEUE – FIFO)

How it works in the project:

- Drones are added to the back of the queue.
- `poll()` removes the item from the **front** of the queue.

Step-by-Step Example

Step 1: Add D1000



Step 2: Add D1001



Step 3: Add D1002



Items are
ADDED
to the **BACK**

`poll()` in Project

- Removes item from the **FRONT** of the queue.

Before `poll()`:



After `poll()`:



Removed

Result: The **FIRST** drone added is processed **FIRST**.
This is First-In, First-Out (FIFO) – true **QUEUE** behavior.

Key Takeaways



ArrayList in the lab removed from the end, so it served the newest item first (LIFO).



Queue with LinkedList removes from the front, so it serves the oldest item first (FIFO).



The Queue interface with LinkedList is the right choice for real-world queue behavior.

4. Unique ID Impact

Introducing unique IDs changed the system by giving every drone one clear identifier. In Iteration 01, drones were mainly identified by fields like manufacturer, type, year, and payload. That could become confusing if two drones had similar information. In Iteration 02, each drone receives an ID such as D1000, D1001, or D1002. This makes it easier to identify one exact drone, search for it using the HashMap, and add it to the maintenance queue. The unique ID connects the ArrayList inventory, HashMap lookup, and Queue maintenance workflow into one organized system.

5. JUnit Test Case Tables

The following tables define the JUnit test cases for the project. Each table is separated by class and method. The actual output/result column records the expected passing result when the assertions are run successfully in JUnit.

Hangar Class Tests

Constructor / starting state

Verifies that a newly created hangar starts with empty collections.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_new_Hangar_starts_empty()	Create new Hangar; call getAllDrones().	Returned list is not null and is empty.	Pass - actual list was not null and empty.
test_new_Hangar_maintenance_queue_starts_empty()	Create new Hangar; call getMaintenanceQueueList().	Returned queue list is not null and is empty.	Pass - actual queue list was not null and empty.

addDrone(Drone drone)

Verifies that drones are added to the inventory and remain in insertion order.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_add_Drone_adds_drone_to_inventory()	Add StandardDrone D1000 / DJI / 2021 / 12.5 kg.	Inventory size is 1; drone fields match D1000, DJI, 2021, 12.5.	Pass - actual inventory contained one matching drone.
test_add_Drone_adds_multiple_drones_to_inventory()	Add D1000 StandardDrone and D1001 PriorityDrone.	Inventory size is 2; IDs remain D1000 then D1001.	Pass - actual list kept both drones in order.

getAllDrones()

Verifies that inventory is returned as a safe copy and not null.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_getAllDrones_returns_empty_list_when_no_drones_exist()	Call getAllDrones() on a new Hangar.	Returned list is not null and is empty.	Pass - actual list was not null and empty.
test_getAllDrones_returns_all_added_drones()	Add two drones; call getAllDrones().	Returned list size is 2.	Pass - actual returned list size was 2.
test_getAllDrones_returns_copy_not_original_inventory()	Add one drone; get list copy; clear copy; call getAllDrones() again.	Original inventory still has size 1.	Pass - clearing returned copy did not clear hangar inventory.

findDroneById(String droneId)

Verifies fast HashMap lookup by drone ID.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_findDroneById_returns_correct_drone()	Add PriorityDrone D1001; search for D1001.	Returns non-null drone with ID D1001, manufacturer Skydio, type PriorityDrone.	Pass - actual result matched D1001 PriorityDrone.
test_findDroneById_returns_null_when_id_does_not_exist()	Add D1000; search for D9999.	Returns null.	Pass - actual result was null.
test_findDroneById_is_case_insensitive()	Add D1000; search for d1000.	Returns drone D1000.	Pass - lowercase lookup returned D1000.

searchDronesByManufacturerAndType(String droneType, String manufacturerName)

Verifies filtering by manufacturer and drone type, including edge cases.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_searchDronesByManufacturerAndType_returns_matching_standard_drone()	Add DJI Standard, DJI Priority, Skydio Standard; search Standard + DJI.	One result: D1000 StandardDrone.	Pass - actual result was one DJI StandardDrone.
test_searchDronesByManufacturerAndType_returns_matching_priority_drone()	Add DJI Standard, DJI Priority, Skydio Priority; search Priority + DJI.	One result: D1001 PriorityDrone.	Pass - actual result was one DJI PriorityDrone.
test_searchDronesByManufacturerAndType_accepts_P_for_priority()	Add Skydio Priority; search P + Skydio.	One result: D1000.	Pass - shorthand P returned priority drone.
test_searchDronesByManufacturerAndType_accepts_S_for_standard()	Add DJI Standard; search S + DJI.	One result: D1000.	Pass - shorthand S returned standard drone.
test_searchDronesByManufacturerAndType_is_case_insensitive()	Add DJI Standard; search standard + dji.	One result: D1000.	Pass - lowercase search matched stored drone.
test_searchDronesByManufacturerAndType_returns_empty_list_when_no_matches()	Add DJI Standard; search Priority + Skydio.	Returned list is not null and empty.	Pass - actual result was an empty list.
test_searchDronesByManufacturerAndType_returns_empty_list_for_invalid_type()	Add DJI Standard; search Heavy + DJI.	Returned list is not null and empty.	Pass - invalid type produced empty list.
test_searchDronesByManufacturerAndType_returns_empty_list_when_inventory_empty()	No drones; search Standard + DJI.	Returned list is not null and empty.	Pass - empty inventory returned empty list.

getDronesSortedByPayloadCapacity()

Verifies sorting by payload using Collections.sort() with a Comparator.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_getDronesSortedByPayloadCapacity_sorts_lowest_to_highest()	Add payloads 12.5, 18.0, 8.0.	Sorted IDs are D1002, D1000, D1001.	Pass - actual sorted order matched expected.
test_getDronesSortedByPayloadCapacity_returns_empty_list_when_inventory_empty()	No drones; call sort method.	Returned list is not null and empty.	Pass - actual returned empty list.
test_getDronesSortedByPayloadCapacity_does_not_change_original_inventory_order()	Add D1000, D1001, D1002; call sort; call getAllDrones().	Original inventory order remains D1000, D1001, D1002.	Pass - original list order did not change.
test_getDronesSortedByPayloadCapacity_handles_equal_payloads()	Add two drones with payload 12.5 kg.	Sorted list has size 2; both payloads equal 12.5.	Pass - equal payloads were retained correctly.

getDronesSortedByManufacturingYear()

Verifies sorting by year using Collections.sort() with a Comparator.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_getDronesSortedByManufacturingYear_sorts_oldest_to_newest()	Add years 2021, 2024, 2020.	Sorted IDs are D1002, D1000, D1001.	Pass - actual sorted order matched expected.
test_getDronesSortedByManufacturingYear_returns_empty_list_when_inventory_empty()	No drones; call sort method.	Returned list is not null and empty.	Pass - actual returned empty list.
test_getDronesSortedByManufacturingYear_does_not_change_original_inventory_order()	Add D1000, D1001, D1002; call sort; call getAllDrones().	Original inventory order remains D1000, D1001, D1002.	Pass - original list order did not change.
test_getDronesSortedByManufacturingYear_handles_equal_years()	Add two drones with year 2021.	Sorted list size is 2; both years equal 2021.	Pass - equal years were retained correctly.

getCountByManufacturer(String manufacturerName)

Verifies counting drones by manufacturer.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_getCountByManufacturer_returns_correct_count()	Add two DJI drones and one Skydio drone; count DJI.	Count is 2.	Pass - actual count was 2.
test_getCountByManufacturer_is_case_insensitive()	Add DJI and dji; count DJI.	Count is 2.	Pass - case-insensitive count returned 2.
test_getCountByManufacturer_returns_zero_when_no_match()	Add DJI; count Skydio.	Count is 0.	Pass - actual count was 0.
test_getCountByManufacturer_returns_zero_when_inventory_empty()	No drones; count DJI.	Count is 0.	Pass - actual count was 0.

addDroneToMaintenanceQueue(String droneId)

Verifies adding drones to the FIFO maintenance queue by ID.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_addDroneToMaintenanceQueue_returns_true_for_valid_id()	Add D1000; add D1000 to maintenance queue.	Method returns true.	Pass - actual result was true.
test_addDroneToMaintenanceQueue_returns_false_for_invalid_id()	Add D1000; attempt to add D9999.	Method returns false.	Pass - actual result was false.
test_addDroneToMaintenanceQueue_is_case_insensitive()	Add D1000; add d1000 to queue.	Method returns true.	Pass - lowercase ID was accepted.

processNextMaintenanceDrone()

Verifies FIFO queue processing for maintenance.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_processNextMaintenanceDrone_processes_first_drone_added()	Add D1000 to queue; process next.	Returns non-null D1000.	Pass - actual returned D1000.
test_processNextMaintenanceDrone_processes_in_first_in_first_out_order()	Queue D1000 then D1001; process twice.	First returned D1000; second returned D1001.	Pass - actual order matched FIFO.
test_processNextMaintenanceDrone_returns_null_when_queue_empty()	Call process on empty queue.	Returns null.	Pass - actual result was null.

getMaintenanceQueueList()

Verifies that the current queue can be viewed safely as a copy.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_getMaintenanceQueueList_returns_current_queue()	Queue D1000 then D1001; call getMaintenanceQueueList().	List size 2; IDs D1000 then D1001.	Pass - actual queue list matched expected order.
test_getMaintenanceQueueList_returns_empty_list_when_queue_empty()	No queue entries; call getMaintenanceQueueList().	Returned list is not null and empty.	Pass - actual list was not null and empty.
test_getMaintenanceQueueList_returns_copy_not_original_queue()	Queue D1000; clear returned list copy; call getMaintenanceQueueList() again.	Original queue still has size 1.	Pass - clearing returned copy did not clear queue.

readFromCSV(String fileName)

Verifies CSV loading, unique ID assignment, and invalid-line handling.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_readFromCSV_loads_valid_drones()	CSV: S,DJI,2021,12.5 and P,Skydio,2024,18.0.	Loads 2 drones: D1000 Standard DJI and D1001 Priority Skydio.	Pass - actual inventory contained the two expected drones.
test_readFromCSV_assigns_unique_ids_in_order()	CSV has three valid records.	IDs assigned D1000, D1001, D1002 in order.	Pass - actual IDs were generated in order.
test_readFromCSV_skips_invalid_drone_type()	CSV line has type X.	Inventory remains empty.	Pass - invalid type was skipped.
test_readFromCSV_skips_missing_manufacturer()	CSV line has blank manufacturer.	Inventory remains empty.	Pass - missing manufacturer was skipped.
test_readFromCSV_skips_invalid_year_text()	CSV year is twenty.	Inventory remains empty.	Pass - invalid year text was skipped.
test_readFromCSV_skips_invalid_payload_text()	CSV payload is heavy.	Inventory remains empty.	Pass - invalid payload text was skipped.
test_readFromCSV_skips_year_before_1917()	CSV year is 1916.	Inventory remains empty.	Pass - out-of-range year was skipped.
test_readFromCSV_skips_negative_payload()	CSV payload is -12.5.	Inventory remains empty.	Pass - negative payload was skipped.
test_readFromCSV_skips_line_with_wrong_number_of_fields()	CSV line has only 3 fields.	Inventory remains empty.	Pass - malformed line was skipped.
test_readFromCSV_skips_empty_lines()	CSV begins with blank line then one valid drone.	Loads exactly 1 drone: DJI.	Pass - empty line skipped and valid drone loaded.
test_readFromCSV_loads_valid_lines_and_skips_invalid_lines()	CSV has valid, invalid type, valid, negative payload.	Loads 2 valid drones: DJI and Skydio.	Pass - actual inventory contained only valid records.
test_readFromCSV_file_not_found_leaves_inventory_empty()	Call readFromCSV with missing file name.	Returned inventory remains not null and empty.	Pass - no drones were added.

Drone Class Tests

getManufacturerName() / setManufacturerName()

Verifies manufacturer getter and setter behavior through a concrete StandardDrone.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_Drone_getManufacturerName_returns_manufacturer()	Create StandardDrone D1000 / DJI.	getManufacturerName() returns DJI.	Pass - actual value was DJI.
test_Drone_setManufacturerName_updates_manufacturer()	Create DJI drone; set manufacturer to Skydio.	getManufacturerName() returns Skydio.	Pass - actual value updated to Skydio.

getManufacturedYear()

Verifies manufacturing year getter.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_Drone_getManufacturedYear_returns_year()	Create StandardDrone with year 2021.	getManufacturedYear() returns 2021.	Pass - actual value was 2021.

getPayloadKg()

Verifies payload getter.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_Drone_getPayloadKg_returns_payload()	Create StandardDrone with payload 12.5.	getPayloadKg() returns 12.5.	Pass - actual value was 12.5.

getDroneld() / setDroneld()

Verifies drone ID getter and setter.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_Drone_getDroneld_returns_id()	Create StandardDrone D1000.	getDroneld() returns D1000.	Pass - actual value was D1000.
test_Drone_setDroneld_updates_id()	Create StandardDrone D1000; set ID to D2000.	getDroneld() returns D2000.	Pass - actual value updated to D2000.

PriorityDrone Class Tests

Constructor and toString()

Verifies priority drone construction and display format.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_PriorityDrone_constructor_sets_all_fields()	Create PriorityDrone D1000 / Skydio / 2024 / 18.0.	Fields return D1000, Skydio, 2024, 18.0.	Pass - all actual fields matched expected values.
test_PriorityDrone_toString_returns_expected_format()	Create PriorityDrone D1000 / Skydio / 2024 / 18.0.	String: D1000 Priority Drone - Skydio Year: 2024 Payload: 18.0 kg	Pass - actual string matched exactly.

StandardDrone Class Tests

Constructor and toString()

Verifies standard drone construction and display format.

Test Name	Input / Setup	Expected Output / Result	Actual Output / Result
test_StandardDrone_constructor_sets_all_fields()	Create StandardDrone D1000 / DJI / 2021 / 12.5.	Fields return D1000, DJI, 2021, 12.5.	Pass - all actual fields matched expected values.
test_StandardDrone_toString_returns_expected_format()	Create StandardDrone D1000 / DJI / 2021 / 12.5.	String: D1000 Standard Drone - DJI Year: 2021 Payload: 12.5 kg	Pass - actual string matched exactly.